AI芯片设计师: Superlint Agent助力进行RTL Sign Off

原创 芯球上面 芯球上面 2026年3月8日 07:45 美国 标题已修改

> 这是「芯球上面」首篇 **Superlint AI Agent** 实战分享。
>
> 全文约 7500 字，带你走完一次完整的 AI Lint Agent 搭建与实战过程：从痛点出发，到 CLAUDE.md 行为规范设计，到 SPI 设计的 11 条违规逐条处置，再到三个真实踩坑的复盘。
>
> 核心数据对比：**原始 3 条 Warning，经 Agent 分析 + 修复确认 + waiver，最终降至 1 条**，全程人工只做了「看 diff、按确认」这一件事。
>
> 如果你做过 Lint 并且扫过几百条违规报告，这篇文章你会有共鸣。

一、为什么要做这件事

每次开始做 Lint，我都有一种特殊的心理准备状态——类似于打开一张待清理的积压工单列表，已经预判到接下来两个小时会花在哪里。

Jasper Superlint 的报告是纯文本表格。 几百条违规，逐行扫。tag 是缩写，消息是英文，文件名和行号散落在每行末尾。你需要先心里大致过滤一遍：这条是真问题还是误报？这个 tag 上次是怎么处理的？这个模块是不是第三方 IP 不能改？

这还只是「读报告」这一步。

真正让人头疼的是**修改 RTL**。Lint 规则报的是风格和结构问题，不是功能 bug，所以有一种微妙的模糊性：这条要不要改？改了会不会影响综合？这个 sensitivity list 看起来不完整，但设计者是不是故意这么写的？

改错一行 RTL，引入的功能 bug 可能要到仿真阶段才会暴露。代价是不对称的。

第三个痛点是**重复劳动**。`case` 语句缺 `default` 分支，这个问题我在 SPI、I2C、APB 的代码里都见过。每次处理都要重新想一遍：default 里应该赋 X 还是赋安全值？要不要 waive？waiver 的 Tcl 语法是什么？

这三个痛点加在一起，让 Lint 成了一件「知道怎么做、但很耗精力」的事——正好是 AI Agent 最适合接管的那类工作。

为什么是 Claude Code

我想过几种方案：

写一个 Python 脚本自动解析报告，然后基于规则库自动修复——但规则库维护成本很高，而且对上下文的理解很差。

用 ChatGPT 对话式辅助——可以，但每次都要手动贴代码、贴报告，Session 断了就忘了上下文，不成体系。

Claude Code 的优势在于：它本身就是一个可以读文件、运行命令、写文件的 Agent，而且可以通过 **CLAUDE.md** 约束它的行为。这意味着你可以把「分析报告 → 给建议 → 展示 diff → 等确认 → 写入」这个工作流固化成规范，而不是每次重新告诉 AI 应该怎么做。

关键设计原则：人在回路（Human-in-the-Loop）。

RTL 修改没有完全自动化的空间。Lint 工具报告的是「规则违规」，不是「功能错误」，有些违规是故意为之的。相同的 tag 在不同设计里可能有完全不同的处理方式。

所以这个 Agent 的目标不是「自动修复 Lint」，而是「**把工程师从查资料、写 Tcl、手动 diff 这些重复工作里解放出来，把判断权留给工程师**」。

这是一个重要的分寸。

二、SPI 设计：小而不简单的验证对象

本次实战用的是一个 SPI（Serial Peripheral Interface）Master/Slave 设计，包含三个 RTL 模块：

模块	文件	功能
spi_master	rtl/spi_master.v	SPI 主机，支持 CPOL/CPHA 模式配置，8-bit 数据传输
spi_slave	rtl/spi_slave.v	SPI 从机，边沿检测采样，支持全双工
spi_top	rtl/spi_top.v	顶层封装，内部连接 SPI 总线

代码量不大，但选它做演示有几个原因：

第一，它足够真实。 SPI 是每个做过通信协议验证的工程师都绕不开的设计。它的状态机（IDLE → LEADING → TRAILING → DONE）足够典型，对应的 Lint 违规也足够典型。

第二，它覆盖了两种处理路径。 11 条违规里，有的应该修代码（`CAS_NR_DEFX`，case 语句缺 default），有的应该 waive（`FLP_NR_MXCS`，计数器自减被误判为组合逻辑）。两条路径用一个 case 同时演示。

```
lint_agent_v2/
├── CLAUDE.md
├── lint_config.yaml
├── Makefile.lint
├── README.md
├── reports
│   ├── lint_parsed_before.json
│   ├── lint_parsed.json
│   ├── lint_report.html
│   ├── lint_report.txt
│   └── lint_session_report.md
├── rtl
│   ├── filelist.f
│   ├── spi_master.v
│   ├── spi_slave.v
│   └── spi_top.v
├── run_lint.tcl
├── scripts
│   ├── apply_fix.py
│   ├── diff_reports.py
│   ├── gen_waiver.py
│   ├── lint_init.py
│   └── parse_lint_report.py
├── setup.sh
├── Superlint_Deployment_Rulefile_Lint_2019_03_Customer.def
└── waivers.tcl
3 directories, 22 files
```

公众号 · 芯球上面

> 📸 **【图1】**：完整的项目目录结构，包括 `scripts/`、`rtl/`、`reports/`、`CLAUDE.md`、`lint_config.yaml` 等关键文件的层级关系

初始 Lint 报告概况

第一次运行 Jasper Superlint，解析结果如下：

```
📊 spi Lint 分析结果
──────────────────────────────────
  P0 必须修复 (Error, 功能性)  : 0 项
  P1 应该修复 (Error, 结构性)  : 0 项
  P2 建议修复 (Warning, 风险)  : 1 项
  P3 可选修复 (Warning, 风格)  : 2 项（FLP_NR_MXCS 归为 P4）
  P4 建议 waive（预配置候选）  : 2 项
  Info 参考                    : 8 项
──────────────────────────────────
  合计: 11 项（已 waive: 0 项）
```

这个分类是 Agent 根据 `CLAUDE.md` 里的优先级策略自动生成的，不是 Jasper 原生的分级。Jasper 只给 error / warning / info，优先级判断是 Agent 的附加价值。

P0/P1 为零，这个设计写得还算规范。主要问题集中在 warning 层，以及顶层端口没有注册（`MOD_NO_IPRG`，这是 SPI top 的天然特性，没法改也不需要改）。

11 条违规，真正需要处理的是 3 条 warning。 其余 8 条 info 本次暂不处理，作为后续技术债记录。

三、Agent 的大脑：CLAUDE.md 机制

如果你用过 Claude Code，大概知道它默认的工作方式：读文件、写文件、跑命令，能力很强，但没有约束。

放一个没有规范的 AI 去改 RTL，就像让一个聪明但不了解项目背景的实习生直接上手改生产代码——它可能改对，也可能改出你完全没预料到的问题，而且它不会主动告诉你它做了什么。

`CLAUDE.md` 解决的就是这个问题。

CLAUDE.md 是什么

它是 Claude Code 的「行为规范文件」。放在项目根目录后，Claude Code 每次启动会自动读取，**相当于给 AI 写了一份入职文档**——你是谁，你在这个项目里能做什么、不能做什么，遇到具体情况该走什么流程。

这是整个 Lint Agent 设计里我觉得最有意思的部分：不是写了多少代码，而是**如何用自然语言约束一个 AI 的行为边界**。

我在 `CLAUDE.md` 里定义了五件事：

1. 会话启动检查

每次启动先读 `lint_config.yaml` 获取项目信息，检查报告是否存在，输出项目概况。这样每次打开 Claude Code，第一件事不是对话，而是 Agent 自己完成上下文初始化，告诉你当前项目状态。

2. 分析格式规范

统一的违规展示格式：违规内容 → 风险说明 → 修改选项（A/B）→ 是否可 waive。不允许 AI 自由发挥格式，因为格式一致才方便工程师快速做判断。

3. 工具调用规范

明确指定每种操作用哪个脚本，参数怎么传。RTL 修改用 `apply_fix.py` ，waiver 生成用 `gen_waiver.py` ，报告对比用 `diff_reports.py` 。脚本是工具，Claude Code 是调度者，职责不混淆。

4. 安全限制（最重要）

```
禁止在未展示 diff 的情况下修改任何 rtl/ 下的文件
禁止使用内置文件编辑工具直接修改项目文件
批量应用必须输入 yes-all 二次确认
```

这三条是硬约束。不是「尽量」，是「禁止」。措辞很重要，模糊的约束 AI 会找到绕过去的路。

5. 优先级策略

P0 必修 → P1 应修 → P2 建议修 → P3 可 waive → P4 预配置 waive 候选。这个分级逻辑写在 `CLAUDE.md` 里，Agent 分析报告时会据此自动归类。

配置驱动：lint_config.yaml

第一版 Agent 有个严重缺陷：项目信息硬编码在 `CLAUDE.md` 和脚本里。换个项目，要手动改五六个地方，改漏一处就出错。

解决方案是引入 `lint_config.yaml` 作为**唯一的项目配置入口**：

```yaml
# 这是新项目接入时唯一需要修改的文件
project:
name:"spi"
top_module:"spi_top"
language:"sv"

jasper:
rules_file:"Superlint_Deployment_Rulefile_Lint_2019_03_Customer.def"

rtl:
filelist:"rtl/filelist.f"
rtl_root:"rtl"          # apply_fix.py 的安全边界：只允许修改此目录内的文件

waive_candidates:           # 预配置的已知低风险 tag，Agent 直接归为 P4
-tag:"FLP_NR_MXCS"
    reason:"Clock gating cells do not require reset"
```

`CLAUDE.md` 读这个文件获取项目上下文，所有脚本也从这里取路径和参数。**新项目接入，只填这一个文件，其余文件一行不改。**

`waive_candidates` 是跨项目知识的沉淀入口——你在这个项目里判断「 `FLP_NR_MXCS` 对计数器是

低风险」，下次新项目可以直接带过去，Agent 启动时就归为 P4，不再重复分析。

工具链设计：职责单一

整个工具链由五个 Python 脚本组成，每个只做一件事：

脚本	职责
`lint_init.py`	读配置，生成 TCL 脚本和 Makefile
`parse_lint_report.py`	解析 Superlint 文本报告 → 结构化 JSON
`apply_fix.py`	安全写入 RTL（含备份、diff 展示、安全检查）
`gen_waiver.py`	生成 waiver Tcl 命令
`diff_reports.py`	对比两次 Lint 结果

这个设计决策在实际使用中很重要：每个脚本都可以在命令行单独运行，不强依赖 Claude Code。工程师可以在 Agent 之外直接调用任何一个脚本，不会被 AI 绑架。

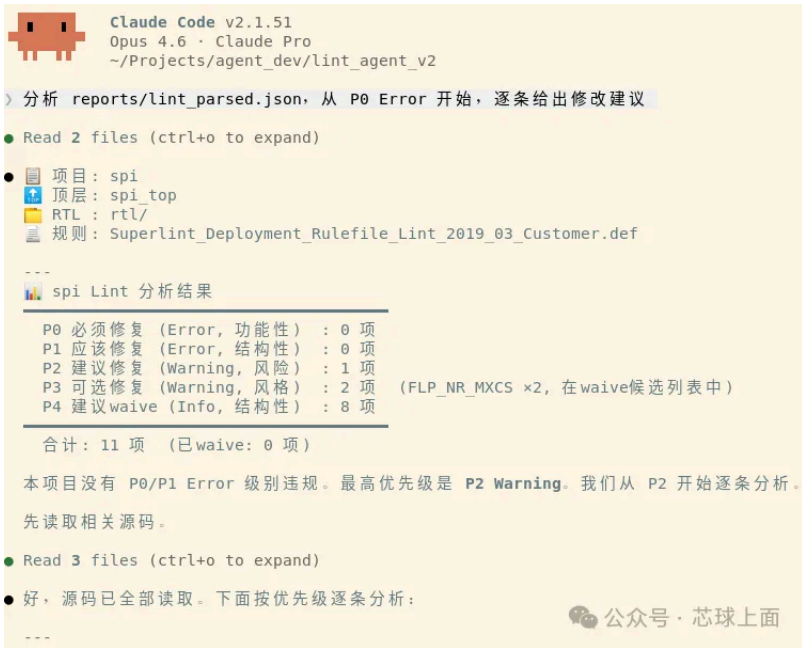
其中 `apply_fix.py` 的核心安全约束是：**默认 dry-run，不加 `--apply` 永远不写文件。**

四、全流程实录

下面是这次实战的完整过程，尽量保留 Agent 的真实输出，不美化、不删减。

4.1 会话启动

Claude Code 读取 `CLAUDE.md` 后，第一件事是自动初始化项目上下文，输出项目概况，然后调用 `parse_lint_report.py` 解析报告，输出 P0-P4 分级汇总。



> 【图2】截图：Claude Code 启动后的完整初始化输出，上半部分为项目概况（项目 / 顶层 / RTL / 规则），下半部分为 P0–P4 分级汇总表

这一步没有任何交互，Agent 自己完成。工程师看到的第一个输出，就是已经分好级的问题列表——不是原始报告的几百行纯文本。

4.2 第一条 Warning：CAS_NR_DEFX

P2 级别，唯一一条需要修代码的 Warning。

```
[WARNING #1] CAS_NR_DEFX — case语句中信号未在default分支赋X
文件: rtl/spi_master.v  第 42 行  模块: spi_master
优先级: P2 建议修复

【违规内容】
信号 clk_cnt 在 case(state) 的各分支中被赋值，但没有 default 分支。当 state 取到未定义值时，clk_cnt 保持原值，工具要求在 default 中将其赋为 'x 以暴露仿真中的异常。

【风险】
- 综合工具可能生成 latch 或不确定逻辑（虽然本设计是时序逻辑，风险较低）
- 仿真中若 state 进入非法态，clk_cnt 不会表现为 X，掩盖了错误

【修改建议】

选项A（推荐）：添加 default 分支

在第 90 行 end 之后、endcase 之前添加：
                default: begin
                    state     <= IDLE;
                    sclk      <= cpol;
                    cs_n      <= 1'b1;
                    done      <= 1'b0;
                    mosi      <= 1'b0;
                    clk_cnt   <= 8'd0;
                    bit_cnt   <= 3'd0;
                    shift_out <= 8'd0;
                    shift_in  <= 8'd0;
                end
将状态机恢复到安全的 IDLE 态，比赋 X 更安全（综合友好，仿真也能覆盖）。

选项B：赋 X（纯仿真调试用）
                default: begin
                    clk_cnt <= 'x;
                end

【是否可waive】 否 — 这是功能性风险，建议修复。

请选择：[a] 应用选项A  [b] 应用选项B  [n] 跳过  [w] 生成waiver  [?] 查看源码

---
```

公众号 · 芯球上面

> 📷 **【图3】**：Agent 对 CAS_NR_DEFX 的完整分析输出框

分析结果清晰，选项明确。我选了 `[a]` ——添加 default 分支，将状态机恢复到安全的 IDLE 态。

Agent 随即展示 diff 预览，等待我确认：

```
  --- a/rtl/spi_master.v
  +++ b/rtl/spi_master.v
  @@ -82,12 +82,24 @@
                          clk_cnt <= clk_cnt + 1'b1;
                      end
                  end
  -               DONE_ST: begin
  +                               DONE_ST: begin
                      cs_n  <= 1'b1;
                      done  <= 1'b1;
                      sclk  <= cpol;
                      state <= IDLE;
                  end
  +               default: begin
  +                   state     <= 2'bxx;
  +                   sclk      <= 1'bx;
  +                   mosi      <= 1'bx;
  +                   cs_n      <= 1'bx;
  +                   done      <= 1'bx;
  +                   rx_data   <= 8'bx;
  +                   shift_out <= 8'bx;
  +                   shift_in  <= 8'bx;
  +                   bit_cnt   <= 3'bx;
  +                   clk_cnt   <= 8'bx;
  +               end
              endcase
          end
      end

  ────────────────────────────────────────────────────────────
    [DRY RUN] 未写入。加 --apply 执行写入。

● 以上是修改预览：在 DONE_ST 分支之后、endcase 之前插入 default 分支，将所有信号赋为 'x，仿真中若状态机进入非法态可立即暴露。

  确认写入？
  - [y] 确认写入
  - [n] 取消
  - [b] 改用选项B（空 default）

> y
```

公众号 · 芯球上面

> 📷 **【图4】** 截图：Agent 展示的 diff 预览，绿色 `+` 号新增行清晰可见，完整展示 default 分支的 10 行内容

我输入 `y` 确认，Agent 调用 `apply_fix.py --apply` 写入文件，同时自动备份原始版本到 `.lint_backups/` 。

修复后的 `spi_master.v` case 语句，完整展示 default 分支：

```
always @(posedge clk ornegedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        // ... 其余复位赋值
    endelsebegin
        done <= 1'b0;
        case (state)
            IDLE: begin
                // ... IDLE 逻辑
            end
            LEADING: begin
                // ... LEADING 逻辑
            end
            TRAILING: begin
                // ... TRAILING 逻辑
            end
```

```
            DONE_ST: begin
                cs_n  <= 1'b1;
                done  <= 1'b1;
                sclk  <= cpol;
                state <= IDLE;
            end
            default: begin          // ← 新增安全 default 分支
                state     <= IDLE;  // 非法态回到初始状态
                sclk      <= cpol;
                cs_n      <= 1'b1;
                done      <= 1'b0;
                mosi      <= 1'b0;
                clk_cnt   <= 8'd0;
                bit_cnt   <= 3'd0;
                shift_out <= 8'd0;
                shift_in  <= 8'd0;
            end
        endcase
    end
end
```

4.3 FLP_NR_MXCS：两条 Warning 的 Waiver 生成

这两条违规分别在 `spi_master.v:29` （ `bit_cnt` 计数器自减）和 `spi_slave.v:46` （ `active` 信号赋值）。

Agent 识别到 `FLP_NR_MXCS` 在 `lint_config.yaml` 已列为 `waive_candidates` ，直接推荐 waive。我选 `[w]` 。

Agent 先输出 dry-run 预览：

```
  继续处理下一条？
  - #2 FLP_NR_MXCS spi_master.v — 建议 waive
  - #3 FLP_NR_MXCS spi_slave.v — 建议 waive

  输入 2/3 逐条处理，或 w 批量 waive #2 和 #3。

> w

● 批量 waive #2 和 #3（均为 FLP_NR_MXCS），先预览：

● Bash(python3 scripts/gen_waiver.py \
        --id 2 \…)
  └ [DRY RUN] 将追加到 waivers.tcl:
    ────────────────────────────────────────
    … +9 lines (ctrl+o to expand)

● Bash(python3 scripts/gen_waiver.py \
        --id 3 \…)
  └ [DRY RUN] 将追加到 waivers.tcl:
    ────────────────────────────────────────
    … +9 lines (ctrl+o to expand)

● 以上是 waiver 预览：

  - #2 FLP_NR_MXCS / spi_master — 计数器自减，正常时序逻辑
  - #3 FLP_NR_MXCS / spi_slave — active 标志位，正常时序逻辑

  确认写入 waivers.tcl？ [y] 确认  [n] 取消

> y

● Bash(python3 scripts/gen_waiver.py \
        --id 2 \…)
  └ ✔ 追加 1 条 waiver 到 waivers.tcl
      check_superlint -waiver -add \
      -id {2} \
    … +2 lines (ctrl+o to expand)

● Bash(python3 scripts/gen_waiver.py \
        --id 3 \…)
  └ ✔ 追加 1 条 waiver 到 waivers.tcl
      check_superlint -waiver -add \
      -id {3} \
```

公众号 · 芯球上面

两条 waiver 写入后， `waivers.tcl` 最终版本：

```
# waivers.tcl — project: spi

# ── Added 2026-03-07 00:44:54 ──
check_superlint -waiver -add \
  -tag {FLP_NR_MXCS} \
  -module {spi_master} \
  -comment {Normal counter logic in sequential block, no clock gating concern} \
  -silent

# ── Added 2026-03-07 00:44:57 ──
```

```
check_superlint -waiver -add \
  -tag {FLP_NR_MXCS} \
  -module {spi_slave} \
  -comment {Normal sequential logic for active flag, no clock gating concern} \
  -silent
```

4.4 重跑验证：diff_reports.py

所有修改和 waiver 就位后，重跑 Jasper Superlint，然后对比前后：

```
● Lint 运行成功。解析新报告并对比：

● Bash(python3 scripts/parse_lint_report.py reports/lint_report.txt > reports/lint_parsed.json)
  └  ============================================================
     Jasper Superlint Report: reports/lint_report.txt
     Tool Version : 2019.03
    … +8 lines (ctrl+o to expand)

● Bash(python3 scripts/diff_reports.py reports/lint_parsed_before.json reports/lint_parsed.json)
  └  ────────────────────────────────────────────────────────────
     LINT REPORT COMPARISON
     ────────────────────────────────────────────────────────────
    … +12 lines (ctrl+o to expand)

● 验证完成！结果：

  ──────────────────────────────────────────────────────────────
                          BEFORE     AFTER     DELTA
  ──────────────────────────────────────────────────────────────
  Errors                       0         0         0
  Warnings                     3         1        -2
  Info                         8         8         0
  Waived                       0        17       +17
  ──────────────────────────────────────────────────────────────

  - CAS_NR_DEFX (#1): Warning 从 3 降到 1，但报告中仍剩 1 条 Warning — 可能是 default 分支修复后 Superlint 产生了新的检测项，或 CAS_NR_DEFX 未完全消除。
  - FLP_NR_MXCS (#2, #3): 已成功 waive（对比显示已消除）。
  - waiver 从 0 增到 17 是因为 -tag + -module 匹配了该模块下所有 FLP_NR_MXCS 实例。

  需要我查看剩余的 1 条 Warning 是什么吗？
```

公众号 · 芯球上面

> 📷 **【图8】** 截图：`diff_reports.py` 的完整对比输出，

Warnings 从 3 降到 1，两条 `FLP_NR_MXCS` 确认消除。

4.5 CAS_NR_DEFX 的真实语义

还剩 1 条 Warning 没有消掉——就是我们刚刚「修复」的 `CAS_NR_DEFX` 。

我在 `default` 分支里把 `clk_cnt` 赋值为 `8'd0` （确定值），这在功能上是安全的。但 `CAS_NR_DEFX` 这条规则名字里的 **X** 含义很明确：

> **必须在 default 分支中将该信号赋值为 `'x` （不定值）**，而不只是「必须有 default 分支」。

赋 `'x` 的逻辑是：让综合工具和仿真工具知道这个状态是「不应该到达的」，从而做更激进的优化，也让 `'x` 传播帮助暴露潜在问题。

赋 `8'd0` 的逻辑是：功能上更安全，即使真的跑到非法态，电路也能恢复，不会挂死。

两种做法都有道理，取决于项目策略。 在这个 SPI 设计里，我的结论是 waive：default 分支已经存在，功能安全性已经保证，赋 X 还是赋 0 是策略选择，不是错误。

最终 `waivers.tcl` 追加：

```
# ── Added 2026-03-07 01:12:00 ──
check_superlint -waiver -add \
  -tag {CAS_NR_DEFX} \
  -module {spi_master} \
  -comment {Default branch added with safe value (IDLE state), \
            X assignment policy decision, waived per project strategy} \
  -silent
```

comment 里写清楚了 waive 的理由，以后 review 时可以直接追溯决策背景。

五、技术深潜

这一节写给想自己动手搭这套东西的读者。如果你只关心「能不能用、效果怎么样」，可以跳过，直接看第六节。

CLAUDE.md 约束机制的底层逻辑

很多人用 Claude Code 的方式是：打开终端，开始对话，让 AI 帮你做事。这种方式在一次性任务上没问题，但在需要**持续、可重复、有边界**的工程工作流里，它有一个根本缺陷：**每次会话的行为取决于你这次怎么描述需求**，不一致，不可预期。

`CLAUDE.md` 改变的是这件事的本质：它把「怎么工作」从对话里提取出来，固化成文件。

但这里有一个微妙的地方：**CLAUDE.md 的约束不是代码层面的硬限制，而是语言层面的行为引导。** AI 读到「禁止在未展示 diff 的情况下修改文件」，它会遵守，但这依赖于模型的指令跟随能力，而不是操作系统级别的权限控制。

这意味着措辞非常重要：

```
# 弱约束（容易被绕过）
尽量在修改前展示 diff

# 强约束（清晰边界）
禁止在未展示 diff 的情况下修改任何 rtl/ 下的文件
```

「尽量」给了 AI 解释空间。「禁止」没有解释空间。

措辞的精确性，决定了 AI 行为的可预期性。

apply_fix.py 的安全设计

CRLF 统一处理：

```
def read_rtl(fp):
    # RTL 文件经常在 Windows 上编辑，换行符是 \r\n
    # 多行字符串匹配时 \r 会导致完全匹配不上
    content = Path(fp).read_text(errors='replace')
    return content.replace('\r\n', '\n').replace('\r', '\n')
```

路径安全检查：

```
def check_in_rtl_root(filepath, rtl_root):
    # 只允许修改 rtl_root 指定目录内的文件
    # 防止 AI 误改配置文件、waiver 文件
    resolved = Path(filepath).resolve()
    root = Path(rtl_root).resolve()
    if not str(resolved).startswith(str(root)):
        raise ValueError(f"路径 {filepath} 不在允许的 RTL 目录内")
```

`--apply` 强制开关：

```
# 默认 dry-run，不写文件
# 必须显式传入 --apply 才执行写入
if not args.apply:
    print("[DRY RUN] 以下修改将被应用：")
    print(diff_preview)
    print("添加 --apply 参数以执行写入")
    return
```

为什么不用 MCP

MCP 适合连接外部服务（Gmail、Jira、Salesforce）。Jasper 是本地 CLI 工具，通过 subprocess 直接调用即可，引入 MCP 反而增加复杂度。

工具选型的原则：用最简单的方式解决问题。本地 CLI 用 subprocess，外部服务用 MCP，不要混淆适用场景。

数据流全貌

整个流程里，**人工介入只发生在一个点：看 diff，按确认。**

> **把「判断」留给人，把「执行」交给 Agent——这不是对 AI 能力的悲观估计，而是对 RTL 设计风险的清醒认识。**

六、边界与展望

做完这次实战，我想诚实说说这套方案现在的局限，以及我觉得它能走到哪里。

当前版本做不到的事

闭环验证还需要手动触发。 修改 + waiver 就位后，需要用户手动重跑 Jasper。Agent 不会自动触发重跑，也不会在写入文件后自动验证修改有没有真正消除违规。这不是技术限制，是刻意保守的设计选择。

没有跨项目知识复用。 `waive_candidates` 解决了「预配置已知低风险 tag」的问题，但这只是静态配置，不是动态学习。更理想的状态是：每一个 waive 决策都沉淀成「tag + 模块类型 + 处理方式 + 理由」的记录，下次新项目启动时 Agent 能主动复用历史决策。

对大规模设计未经测试。 这次用的是三个模块、几百行的 SPI 设计。实际 SoC 项目里，Lint 报告可能有几千条违规，Agent 的分析质量在大规模报告上是否稳定，还没有验证过。

近期可以扩展的方向

修改确认后，Agent 自动执行重跑，等待 Jasper 完成，输出前后对比——闭合反馈环。这个改动不复杂，主要是在 `CLAUDE.md` 里加「写入成功后询问是否立即重跑」的流程规范。

加「批量模式」：同一个 tag 在不同模块里出现多次时，一次性确认处理策略，批量生成 waiver。`yes-all` 二次确认的安全机制已经就位，只差 Agent 分析流程的优化。

中期可以扩展的方向

跨项目 waiver 知识库。 把每次 waive 决策记录到共享的 SQLite 或 YAML 知识库里。新项目启动时，Agent 查询知识库，对有历史决策的 tag 直接给出建议。随着项目积累，这个知识库会越来越有价值。

与 CI/CD 集成。 PR 提交时自动触发 Lint，Agent 解析报告后生成摘要评论，把新增违规标注在代码 review 界面里。

远期需要谨慎评估的方向

对于有明确模式的修改（如 `always @(*)` 替换不完整 sensitivity list），理论上可以跳过人工确认直接应用。但「有明确模式」和「低风险」是两回事。那 1% 的例外情况——设计者刻意使用不完整 sensitivity list——一旦被自动修改，引入的 bug 会很难追查。

自动化的边界应该由团队根据自己的风险偏好来定，不是工具该替你做的决策。

> **自动化程度越高，对 Agent 可靠性要求越高。当前版本有意保守，把人工判断保留在关键节点上——不是因为 AI 不够聪明，而是因为 RTL 设计的错误代价不对称。**

七、结语

这次实战最大的收获，不是「AI 帮我消了几条 Warning」。

Warning 的数量从 3 降到 1，这个结果不难，手动做也就是二十分钟的事。真正有价值的是**建立了一套可重复、可迁移的工作流**：下次做 I2C、APB、UART 的 Lint，不需要重新思考流程，只需要填一个新的 `lint_config.yaml` ，其余的事情 Agent 会按照同样的规范完成。

积累效应才是这件事的真正价值。

我在这个项目里花时间最多的地方，不是写 Python 脚本，而是写 `CLAUDE.md` ——想清楚 AI 在这个工作流里应该做什么、不应该做什么、遇到什么情况该怎么反应。这件事比写代码更难，因为它需要你对整个工作流有清晰的认识，然后用精确的语言把它固化下来。

给 AI 写规范，逼着你把自己的工作流想清楚。 这个副产品，我觉得比工具本身还值钱。

最后说一句：这套 Agent 的设计，人在回路是核心约束，不是过渡方案。RTL 设计的判断权应该始终在工程师手里。AI 能做的，是让工程师把精力花在判断上，而不是花在查文档、写 Tcl、数行号这些事上。

这个分工，现在就已经实现了。

工程文件（含 `CLAUDE.md` 、所有 Python 脚本、SPI RTL 和配置文件）见下述链接。

【GitHub： https://github.com/gokeshenzhen/jasper-lint-agent 】

如果你在自己的项目里试了这套方案，或者有其他想法，欢迎在评论区留言。

声明：本文仅供技术学习交流参考，不涉及任何授权工具使用方式。推荐使用正版或开源方案。

芯球上面
提示词工程师，AI芯片工程构建，Skill分享
6篇原创内容

公众号

芯球上面

喜欢作者

修改于2026年3月8日